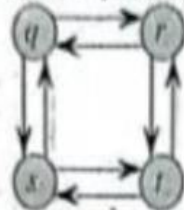


1. The insertion sort algorithm is given below.
 Insertion-Sort (A) // A is an array
 1. for $j = 2$ to A.length
 2. key = A[j]
 3. $i = j - 1$
 4. while ($i > 0$) and ($A[i] > \text{key}$)
 5. $A[i+1] = A[i]$
 6. $i = i - 1$
 7. $A[i+1] = \text{key}$
 - i) How many times line 4 will execute if run on the array $A = (15, 2, 4, 6, 7, 9, 11)$
 - ii) Suppose you want to sort only the first four elements of the above array A. Where should you change in the given Insertion-Sort algorithm?
- b) Suppose you have two algorithms called Algorithm A and Algorithm B to solve a problem. Algorithm A runs faster but sometimes gives incorrect answer. But, Algorithm B runs slower than Algorithm A. Which algorithm should you use and why?
- c) Consider the following two functions.
 Goofy(n)
 1. for $j = 1$ to n
 2. for $k = 1$ to j
 3. print j+k
 Tweety(n)
 1. $j = 1$
 2. while $j \leq n$
 3. for $k = 1$ to j
 4. print j+k
 5. $j = j * 2$
 Now, answer the following questions.
 - a. How many times line 2 of the function Goofy will execute?
 - b. How many times line 2 and 3 of the function Tweety will execute?
 - c. Is it correct to write $\text{Goofy}(n) = \Theta(\text{Tweety}(n))$? Explain your answer.
2.
 - a) Merge-sort runs faster than Quick-sort when run on sorted input. Why?
 - b) Suppose we have an array A where all the elements are same. Can it be considered a Max-Heap? Can it be considered a Min-Heap?
 - c) Illustrate the operation of HEAP-EXTRACT-MAX on the heap
 $A = (15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1)$
 - d) The following elements are given for sorting using quick sort. How the elements will be rearrange when partitioning is done for the first time using last element as a pivot data.
 $A = (14, 19, 54, 20, 65, 37, 17, 85, 95, 32, 45)$
 - e) In an n-element heap, how many nodes are leaf node? How many are non-leaf nodes?

3.

- a) A problem should have optimal substructure if want to solve it by dynamic programming. What is optimal substructure? Taking the following graph as an example, show that 'longest simple path' problem does not have optimal substructure property.



- b) Given a sequence $p = (4, 5, 4, 3, 2)$ which represents the chain of matrices such that i -th matrix A_i is of dimension $p_{i-1} \times p_i$.

m

	1	2	3	4
4			24	0
3		60	0	
2	80	0		
1	0			

s

	1	2	3
4			3
3		2	
2	1		

- i) Fill up the above m and s tables for computing the cost of matrix chain multiplication using dynamic programming for the matrix chain A. 4
- ii) Find an optimal parenthesization of matrix chain product for above matrices. 1
- c) Write down the sequences of four steps to develop a dynamic programming algorithm. 2
- 4.
- a) Show that $4n^3 + 5n \neq \Theta(n^2)$. 2.5
- b) Running time of an algorithm can be expressed by the following recurrence. 2.5
- $$T(n) = 4T(n/16) + \sqrt{n}$$
- Express $T(n)$ in Θ notation.
- c) Recursively express the length of the longest common subsequence of two sequences. 1
- d) Find a longest common subsequence of the following two sequences by filling up the table for a dynamic programming algorithm. 4
- $X = (a, b, a, d, c, b)$ and $Y = (b, c, a, d, c)$

Spring 2016

Ans to the question (1)

(a)

(i)

**How many times line 4 will execute run on the array
A=(15,2,4,6,7,9,11)**

ans:

```
InsertionSort(A)
1. for j = 2 to length(A)
2.   key = A[j]
3.   i = j - 1
4.   while i > 0 and A[i] > key
5.     A[i + 1] = A[i]
6.     i = i - 1
7.   A[i + 1] = key
```

Given the array $A = [15, 2, 4, 6, 7, 9, 11]$, the loop in line 1 will execute 6 times (for $j = 2, 3, 4, 5, 6$, and 7). The number of times the inner while loop in line 4 will execute depends on the initial order of the array. In this case, the inner loop will execute as follows:

For $j = 2$: Inner loop executes 1 time
For $j = 3$: Inner loop executes 2 times
For $j = 4$: Inner loop executes 3 times
For $j = 5$: Inner loop executes 4 times
For $j = 6$: Inner loop executes 5 times
For $j = 7$: Inner loop executes 6 times
Adding up these counts: $1 + 2 + 3 + 4 + 5 + 6 = 21$.

So, the inner loop in line 4 will execute 21 times in total when sorting the array A using the provided insertion sort algorithm.

(II)

Suppose you want to sort only the first four elements of the above array A. Where should you change in the given Insertion-Sort algorithm

?

Ans:

To sort only the first four elements of the array using the Insertion Sort algorithm, we need to make a small modification. In the standard algorithm, iterate from the second element ($j = 2$) to the end of the array. Since it sort only the first four elements, Here's the modified version of the algorithm

```
InsertionSort(A)
1. for j = 2 to 4 // Only iterate for the first four elements
2. key = A[j]
3. i = j - 1
4. while i > 0 and A[i] > key
5. A[i + 1] = A[i]
6. i = i - 1
7. A[i + 1] = key
```

In this modified version, the loop runs from $j = 2$ to 4, ensuring that only the first four elements are sorted. The rest of the algorithm remains the same as the standard Insertion Sort algorithm

(b)

Suppose you have two algorithms called Algorithm A and Algorithm B to solve a problem. Algorithm A runs faster but sometimes gives incorrect answer. But, Algorithm B runs slower than Algorithm A. Which algorithm should you use and why?

Ans:

The choice between Algorithm A and Algorithm B depends on the specific requirements of the problem and the importance of accuracy and speed

Algorithm A: Faster but Sometimes Incorrect

- Pros:
 - Faster execution: Algorithm A runs faster, which could be crucial for time-sensitive applications or large datasets.
- Cons:
 - Incorrect answers: Since Algorithm A can sometimes provide incorrect answers, it might not be suitable for applications where accuracy is critical.

Algorithm B: Slower but Reliable

- Pros:
 - Reliable results: Algorithm B consistently provides correct answers, making it suitable for applications where accuracy is of utmost importance.
- Cons:
 - Slower execution: Algorithm B is slower, which might not be ideal for time-sensitive tasks or scenarios where efficiency is crucial.

Ultimately, the decision depends on the balance between accuracy and speed that problem demands. If accuracy is paramount, even at the cost of speed, Algorithm B is the better choice. If speed is a critical factor and can manage occasional inaccuracies, Algorithm A might be more suitable. It's essential to consider the trade-offs and choose the algorithm that aligns with problem's specific needs and goals

(c)

Consider the following two functions Goofy(n)

```

1. for j= 1 to n
2 . for k=1 to j
3. print j+k
Tweety(s) 1. j=1
2. while i <n
3. for k= 1 to j
4. print j+k
5. j=j*2
  
```

(A)

Now, answer the following questions How many times line 2 of the function Goofy will execute?

Ans:

In the `Goofy(n)` function, there are two nested loops. The outer loop runs from $j = 1$ to n , and the inner loop runs from $k = 1$ to j . The number of times line 2 of the function `Goofy` will execute depends on the iteration of these loops.

For each value of j , the inner loop will execute j times. Therefore, to find out how many times line 2 of the function `Goofy` will execute, we need to sum up the iterations for each value of j from 1 to n :

$$1 + 2 + 3 + \dots + n = n*(n+1)/2$$

So, line 2 of the function `Goofy` will execute $n*(n+1)/2$ times. This is because for each value of j , the inner loop will run j times, and this sum will give you the total number of times line 2 is executed in the entire function

(B)

How many times line 2 and 3 of the function `Tweety` will execute?

Line 2: The `while` loop condition is $i < n$. Since i is not defined or initialized, it's difficult to provide a specific answer without additional context. If i starts at some value and is updated within the loop, we would need that information to determine the number of iterations.

Line 3: If the `while` loop condition is satisfied and i is incremented or modified within the loop, the nested `for` loop will run.

Without the information about the initial value of i , its updates within the loop, and the relationship between i and n , it's not possible to give a precise answer about the number of times lines 2 and 3 will execute.

(C)

Is it correct to write `Goofy(n)=0 (Twenty(n))`? Explain your answer

No, it is not correct to write `Goofy(n) = Twenty(n)`. The two functions, `Goofy(n)` and `Tweety(n)`, are different in terms of their structure, execution, and the values they produce. Let's break down the differences between the two functions:

`Goofy(n)`:

- Contains two nested loops.
- Prints the sum of j and k for each iteration of the nested loops.
- The number of printed values increases as n increases due to the nested loops.

Tweety(s):

- Contains a `while` loop and a nested `for` loop.
- Prints the sum of j and k for each iteration of the nested loop.
- The value of j doubles in each iteration of the outer loop.

The functions `Goofy(n)` and `Tweety(s)` have different loop structures and execution patterns. Simply writing `Goofy(n) = Tweety(n)` implies that both functions are equivalent and produce the same results, which is not the case. The functions have distinct behavior due to the differences mentioned above.

2
(a)

merge sort runs faster than quick sort when run on sorted input. Why?

Merge Sort: Merge Sort is a divide-and-conquer algorithm that works by breaking down the input array into smaller sub-arrays, sorting them, and then merging them back together. When dealing with a sorted input array, Merge Sort still needs to divide the array into smaller sub-arrays, but the merging step becomes more efficient.

In the merging step of Merge Sort, two sub-arrays are combined to create a sorted merged array. When the input is already sorted, these sub-arrays are also already sorted or nearly sorted. This means that the merging process involves minimal comparisons and swapping, resulting in a faster overall execution time. The time complexity of Merge Sort remains relatively consistent regardless of whether the input is sorted or not.

Quick Sort: Quick Sort is another divide-and-conquer algorithm that works by selecting a 'pivot' element and partitioning the array into two sub-arrays: elements less than the pivot and elements greater than the pivot. These sub-arrays are then recursively sorted. However, when the input is already sorted, Quick Sort can perform poorly due to the choice of pivot.

If the pivot chosen is the smallest or largest element in the sorted array, each partitioning step will result in one sub-array being empty and the other containing all elements except the pivot. This leads to an unbalanced partitioning, causing the algorithm to degenerate into essentially a linear search for each partitioning step. This significantly increases the number of comparisons and swaps needed, making Quick Sort much slower than Merge Sort for sorted inputs.

In summary, Merge Sort tends to be more consistent in its performance regardless of input order due to its efficient merging step. Quick Sort's performance heavily relies on pivot selection, and when dealing with sorted input, it can degrade into an inefficient algorithm, making Merge Sort the better choice in such cases.

(b)

suppose you have array A where all the elements are same. can it be considered a Max Heap? can it consider a Min-Heap?

A Max Heap and a Min Heap are binary trees that satisfy the heap property, which states that for a Max Heap, each parent node has a value greater than or equal to the values of its children, and for a Min Heap, each parent node has a value less than or equal to the values of its children.

In the case where all elements in an array are the same, regardless of the value, it can be both a Max Heap and a Min Heap, but it's important to consider the context and how the heap property is interpreted:

Max Heap: Since all elements are the same, every parent node will have the same value as its children. This satisfies the heap property for a Max Heap (parent value greater than or equal to child values), as the value of each parent node is equal to the value of its children. However, it's not a very interesting Max Heap since there's no ordering of elements beyond equality.

Min Heap: Similarly, since all elements are the same, every parent node will again have the same value as its children. This also satisfies the heap property for a Min Heap (parent value less than or

equal to child values), but again, it's not particularly useful for sorting or other purposes due to the lack of ordering.

In practice, heaps are often used to maintain order in a collection of elements, and they are valuable because of their efficient insertion and removal operations. When all elements are the same, the heap's structure can be maintained, but the ordering that makes heaps useful is essentially absent.

So, while an array with all elements the same can technically be considered both a Max Heap and a Min Heap, it doesn't capture the essence or utility of these data structures in terms of sorting or maintaining ordering.

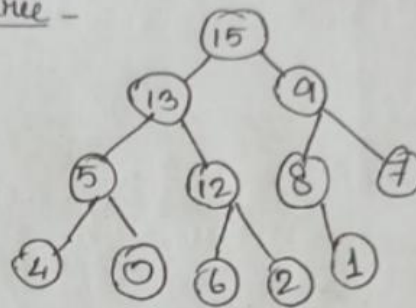
(C)

Answer to question no. 02 (c)

$$A = \{15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1\}$$

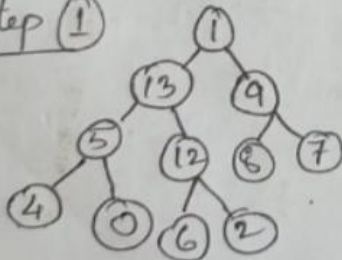
Solve:

Binary tree -

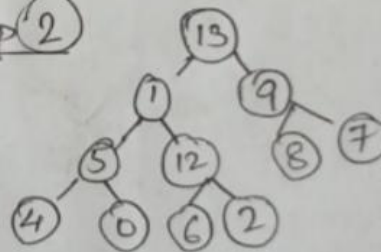


Extract
Max heap

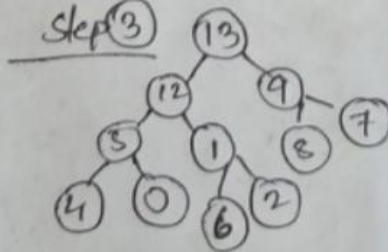
Step 1



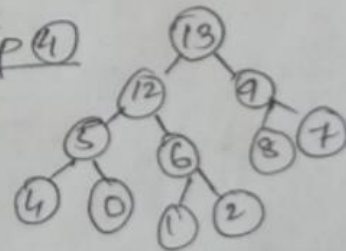
Step 2



Step 3



Step 4



So while an array with all elements the same can technically be considered both a max

(e)

In an n -element heap, how many nodes are leaf node? how many are non-leaf nodes?

In a binary heap (or heap) data structure, there are two types of nodes: leaf nodes and non-leaf nodes.

Leaf Nodes: Leaf nodes are the nodes that do not have any children. In a binary heap, these are the nodes at the lowest level of the heap. Since a binary heap is a complete binary tree, the last level may not be completely filled, so not all nodes at the lowest level will have both children. The maximum number of leaf nodes in a binary heap of size n occurs when the last level is half-filled. If n is an even number, then the number of leaf nodes is $n/2$. If n is an odd number, then the number of leaf nodes is $(n+1)/2$.

Non-Leaf Nodes: Non-leaf nodes are the nodes that have at least one child. These nodes are present above the lowest level of the heap. The number of non-leaf nodes can be calculated by subtracting the number of leaf nodes from the total number of nodes (n).

To summarize:

- Number of leaf nodes in a binary heap with n elements:
 - If n is even: $n/2$
 - If n is odd: $(n+1)/2$
- Number of non-leaf nodes in a binary heap with n elements: $n - \text{number of leaf nodes}$

Remember that this calculation assumes you're dealing with a binary heap. If you're considering a different type of heap structure, the number of leaf nodes and non-leaf nodes might vary.

Ans to the ques No - (4a)

Q Show that, $4n^3 + 5n \neq \theta(n^2)$

For the purpose of contradiction, let us assume that

$$4n^3 + 5n = \theta(n^2)$$

This implies that there exist positive numbers c_1, c_2 and n_0 such that,

$$0 \leq c_1 n^2 \leq 4n^3 + 5n \leq c_2 n^2 \text{ for all } n > n_0$$

so, dividing by n^2 , we get

$$c_1 \leq 4n + \frac{5}{n} \leq c_2$$

The relation on left side i.e. $c_1 \leq 4n + \frac{5}{n}$ holds for large value of n .

But the relation on right side i.e. $4n + \frac{5}{n} \leq c_2$ doesn't hold for large value of n since c_2 is a constant.

So, our assumption is not true at all. Hence it is provided by contradiction that

$$4n^3 + 5n \neq \theta(n^2)$$

(Q1) Ans to the ques No-4(b)

Given that,

$$T(n) = 4T(n/16) + \sqrt{n}$$

Here,

$$a = 4,$$

$$b = 16,$$

$$k = 1/2,$$

$$p = 0$$

$$b^k = 16^{1/2} = 4, \therefore a = b^k$$

$$\text{since, } p = 0 \text{ i.e. } p > -1, T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$$

$$= \Theta(n^{\log_{16} 4} \log^{0+1} n)$$

$$= \Theta(n^{\log_{16} \sqrt{16}} \log n)$$

$$= \Theta(n^{\log_{16} 16^{1/2}} \log n)$$

$$= \Theta(n^{1/2 \log_{16} 16} \log n)$$

$$= \Theta(\sqrt{n} \log n)$$

$$\therefore T(n) = \Theta(\sqrt{n} \log n)$$

4 (C)

The length of the Longest Common Subsequence (LCS) of two sequences can be expressed recursively using dynamic programming. Let's denote the two sequences as A and B, and the lengths of these sequences as n and m respectively.

Let $LCS(i, j)$ be the length of the LCS of the first i elements of sequence A and the first j elements of sequence B.

The recursive formula for calculating $LCS(i, j)$ can be defined as follows:

css

Copy code

```
LCS(i, j) = 0          if i = 0 or j = 0
           LCS(i-1, j-1) + 1  if A[i-1] = B[j-1]
           max(LCS(i-1, j), LCS(i, j-1))  if A[i-1] != B[j-1]
```

Explanation:

If either of the sequences is empty ($i = 0$ or $j = 0$), then the LCS length is 0.

If the last elements of both sequences match ($A[i-1] = B[j-1]$), then we can extend the LCS by including this common element, so we add 1 to the LCS length of the remaining subsequences ($LCS(i-1, j-1)$).

If the last elements do not match, we have two options:

Exclude the last element of sequence A and find the LCS length for the rest of A and B ($LCS(i-1, j)$).

Exclude the last element of sequence B and find the LCS length for the rest of A and B ($LCS(i, j-1)$). We take the maximum of these two options.

To find the length of the overall LCS of sequences A and B, you can call $LCS(n, m)$.

Keep in mind that this recursive approach can lead to overlapping subproblems, so it's often better to use dynamic programming (like memoization or tabulation) to avoid redundant calculations and improve efficiency.

Answer to the question No-4 (d)

Given,

$$X = (a, b, a, d, c, b) \text{ \& } Y = (b, c, a, d, c)$$

$\begin{matrix} y \rightarrow \\ x \downarrow \end{matrix}$	0	b	c	a	d	c
0	0	0	0	0	0	0
a	0	$\leftarrow 0$	$\leftarrow 0$	$\nwarrow 1$	$\leftarrow 1$	$\leftarrow 1$
b	0	$\nwarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$
a	0	$\uparrow 1$	$\leftarrow 1$	$\nwarrow 2$	$\leftarrow 2$	$\leftarrow 2$
d	0	$\uparrow 1$	$\leftarrow 1$	$\uparrow 2$	$\nwarrow 3$	$\leftarrow 3$
c	0	$\uparrow 1$	$\nwarrow 2$	$\leftarrow 2$	$\uparrow 3$	$\nwarrow 4$
b	0	$\nwarrow 1$	$\uparrow 2$	$\leftarrow 2$	$\uparrow 3$	$\nwarrow 4$

$\therefore$ b a d c

Here we find the length 4. Ans:-

